

گزارش کار پروژه شماره 2 - معماری کامپیوتر

اعضای گروه: امیرحسین ربیعیان 403105963، سید ماهد عبدی 403106295، نگین طیبی 403110733

● مقدمه

1.1 چالش پردازنده‌های اسکالر در پردازش داده‌های حجیم در پردازنده‌های استاندارد و معمولی (اسکالر)، پردازش کردن آرایه‌های بزرگ و حجم زیادی از داده‌ها همیشه به دردر بزرگ بوده. مشکل اینجاست که برای انجام به عملیات مشخص روی کلی داده، پردازنده مجبوره وارد حلقه‌های تکراری و طولانی بشه. یعنی سیستم باید برای تک‌تک عناصر داده، به سری دستور ثابت رو دوباره و دوباره واگشی (Fetch) و اجرا کنه. این تکرارهای مداوم باعث می‌شه هم زمان زیادی هدر بره (جریمه زمانی) و هم منابع سخت‌افزاری بی‌دلیل اشغال بشن، که در نهایت بازدهی سیستم رو پایین میاره.

1.2 راهکار پیشنهادی: بهره‌گیری از پردازش برداری (VPU) برای اینکه این مشکل رو دور بزیم و سرعت و توان پردازشی رو حسابی بالا ببریم، بهترین راهکار اضافه کردن به واحد محاسبات برداری (VPU) به پردازنده‌ست. توی این پروژه، ما این ارتقا رو روی به پردازنده بر پایه معماری RISC-V RV32I (پردازنده تمرین 7) انجام دادیم. این واحد برداری مثل به ماژول سخت‌افزاری کاملاً مستقل کنار پردازنده اصلی قرار می‌گیره و از طریق به مسیر ارتباطی مشخص باهاش در ارتباطه. با اضافه شدن VPU، رویکرد سیستم عوض می‌شه؛ یعنی پردازنده اصلی (CPU) دیگه خودش رو درگیر حلقه‌ها نمی‌کنه و فقط کافیه یک دستور برداری رو به همراه آدرس‌های اولیه صادر کنه. به محض اینکه دستور به VPU رسید و سیگنال شروع فعال شد، این واحد به‌صورت مستقل وارد حالت مشغول می‌شه و کار رو دست می‌گیره. VPU خودش داده‌ها رو از حافظه می‌خونه و عملیات رو به‌صورت موازی یا خطی روی کل اون داده‌ها اعمال می‌کنه. وقتی هم کارش تموم شد، به سیگنال اتمام به پردازنده اصلی می‌فرسته تا خبر بده عملیات موفقیت‌آمیز بوده. با این روش، چون دیگه نیازی به خوندن و اجرای دستورات تکراری نیست و کارها موازی پیش می‌ره، سرعت اجرای برنامه‌ها خیلی بیشتر می‌شه.

● معماری سخت افزار و طراحی ماژول ها

2.1 طراحی بانک ثبات‌های برداری مستقل برای اینکه VPU بتونه سریع و مستقل کار کنه، ما یک بانک اختصاصی شامل ۸ ثبات برداری طراحی کردیم. مهم‌ترین نکته اینه که دسترسی به این ثبات‌ها فقط در اختیار خود VPU هست و پردازنده اصلی اصلاً دسترسی مستقیمی بهشون نداره تا جلوی هرگونه تداخل گرفته بشه. هر کدوم از این ثبات‌ها می‌تونن چندین عنصر ۳۲ بیتی رو ذخیره کنن و طول بردارها هم در سیستم قابل تنظیمه. توی سخت‌افزار ما، این بانک به شکل یک آرایه ۲۰۴۸ تایی پیاده‌سازی شده. منطقش هم اینه که ما ۸ ثبات داریم (با ۳ بیت آدرس) که هرکدوم تا ۲۵۶ عنصر (با ۸ بیت آدرس) رو نگهداری می‌کنن. VPU با ترکیب این آدرس‌ها، دقیقاً می‌دونه کدوم عنصر رو باید پردازش کنه.

```
reg [31:0] vector_registers [0:2047];
wire [31:0] fpu_in_a = vector_registers[{alu_vs1, alu_counter[7:0]}];
wire [31:0] fpu_in_b = vector_registers[{alu_vs2, alu_counter[7:0]}];
```

2.2 برای اینکه VPU بتونه بدون دخالت مستقیم پردازنده کار کنه، یک کنترل‌کننده حافظه اختصاصی داخلش قرار دادیم. کار این بخش اینه که آدرس‌های حافظه رو برای خوندن متوالی آرایه‌ها (vload) یا ذخیره کردنشون (vstore) به صورت خودکار تولید کنه.

```

if (mem_opcode == 6'b000010) begin
    MemRead = 1;
    MemAddr = mem_base_addr + (mem_counter << 2);
end else if (mem_opcode == 6'b000011) begin
    MemWrite = 1;
    MemAddr = mem_base_addr + (mem_counter << 2);
    MemDataOut = vector_registers[{mem_vd, mem_counter[7:0]}];
end

```

از اونجایی که CPU و VPU هر دو از یک حافظه مشترک استفاده می‌کنن، باید جلوی تداخل و بن‌بست سیستم رو می‌گرفتیم. برای همین، یک سیستم مدیریت هوشمند (Arbiter) طراحی کردیم. این سیستم به این شکل کار می‌کنه که اگر VPU در حال انتقال داده باشه، اولویت دسترسی به حافظه با اونه و اگر پردازنده اصلی در همون لحظه درخواست کار با حافظه رو داشته باشه، موقتاً متوقف می‌شه تا داده‌ها خراب نشن. (Stall)

```

assign DataAddr = VPU_MemReq_signal ? VPU_MemAddr : M_ALU_res;
assign DataWrite = VPU_MemReq_signal ? VPU_MemWrite : (M_MemWrite & M_Valid & ~stall_M);
assign DataOut = VPU_MemReq_signal ? VPU_MemDataOut : M_Rt_val;

```

2.3) نمودار معماری سامانه (CPU-VPU)

در این معماری، پردازنده اصلی و واحد برداری به عنوان دو ماژول مجزا اما هماهنگ کار می‌کنند. نمودار کلی سیستم شامل سه بخش اصلی است:

۱. پردازنده (CPU): که دستورات را رمزگشایی کرده و اطلاعات لازم را به VPU می‌فرستد.
 ۲. واحد برداری (VPU): که شامل سه زیربخش «کنترل‌کننده»، «بانک ثبات‌های برداری» و «واحد محاسبه (ALU/FPU)» است.
 ۳. حافظه مشترک (Data Memory): که هر دو واحد از طریق یک انتخابگر (Arbiter) به آن متصل هستند.
- ارتباط بین پردازنده و VPU از طریق یک سری سیگنال کنترلی انجام می‌شود؛ CPU سیگنال Start، آدرس‌ها و نوع عملیات را می‌فرستد و در مقابل، VPU سیگنال‌های وضعیتی مثل Busy (مشغول بودن) و Done (اتمام کار) را به پردازنده برمی‌گرداند. (تصاویر ارتباط میان CPU و VPU به علت حجم زیاد در همین پوشه ی report به صورت جداگانه قرار گرفته اند.)

```

VPU_Core vpu_inst (
    .Clk(Clk),
    .Rst(Rst),
    .VPU_Start(VPU_Start),
    .BaseAddr(VPU_BaseAddr),
    .VectorLength(VPU_VectorLength),
    .Opcode(VPU_Opcode),
    ...
)

```

• همگام سازی و مدیریت تداخل ها

3.1) قراردادهای ارتباطی

برای اینکه پردازنده اصلی و VPU بتوانن بدون مشکل با هم کار کنن، یک کانال سخت‌افزاری دقیق برای ارسال پیام‌ها و دریافت وضعیت بینشون تعریف کردیم.

روال کار به این شکله که پردازنده اطلاعات اولیه مثل طول بردار، آدرس حافظه و نوع دستور (Opcode) رو آماده می‌کنه و در نهایت سیگنال Start رو برای VPU فعال می‌کنه. یه نکته کلیدی در طراحی مدار ما اینه که روند دریافت این مقادیر کاملاً مستقل از سیگنال بازنشانی (Rst) عمل می‌کنه؛ یعنی دقیقاً وقتی سیگنال Start یک می‌شه و لبه‌ی کلاک (Clock Trigger) می‌خوره، مقادیر با موفقیت توی ثبات‌های کنترلی VPU بارگذاری شده و عملیات شروع می‌شه.

```

wire VPU_Start = E_is_vector_reg & E_Valid & ~stall_M;

```

در سمت مقابل، VPU هم برای اینکه پردازنده رو در جریان کار قرار بده، از سیگنال‌های وضعیت استفاده می‌کنه. وقتی در حال انجار کاره سیگنال Busy رو می‌فرسته و وقتی کارش تموم شد، با سیگنال Done به پردازنده خبر می‌ده.

```
if (VPU_Start && VectorLength > 0) begin
    if (is_mem_inst && !mem_busy) begin
        mem_busy <= 1;
        mem_counter <= 0;
    end
end
...
```

3.2) پایش وضعیت و کنترل خط لوله

برای اینکه سیستم بهینه‌تر کار کنه و هیچ دستوری از دست نره، پردازنده اصلی دائماً وضعیت VPU رو بررسی می‌کنه. واحد برداری با ارسال سیگنال‌های Busy (درگیر بودن حافظه یا ALU) به پردازنده اعلام می‌کنه که در حال حاضر ظرفیت پذیرش دستور جدید رو داره یا نه.

حالا اگر VPU مشغول باشه و در همین حین به دستور برداری جدید وارد خط لوله پردازنده بشه، چه اتفاقی می‌افته؟ اینجا ما مکانیزم «توقف هوشمند» (Stall) رو پیاده‌سازی کردیم. پردازنده با تشخیص این وضعیت، خط لوله خودش رو در مرحله رمزگشایی متوقف می‌کنه تا کار VPU تموم بشه و به حالت آماده (Ready) برگرده.

```
wire vpu_busy_for_D = (D_is_vpu_mem & VPU_Mem_Busy) |
                      (D_is_vpu_alu & VPU_ALU_Busy) |
                      (D_is_vsetvl & (VPU_Mem_Busy | VPU_ALU_Busy));

wire vpu_stall_req = (D_is_vector | D_is_vsetvl) &
                    (vpu_busy_for_D | (E_is_vector_reg & E_Valid));
```

علاوه بر این، برای جلوگیری از بن‌بست (Deadlock) در دسترسی به حافظه مشترک، یک قانون سفت‌وسخت در سخت‌افزار گذاشتیم: اگر VPU در حال خواندن یا نوشتن آرایه‌ها در حافظه باشه و همزمان پردازنده اصلی هم بخواد به حافظه دسترسی پیدا کنه، اولویت مطلق با VPU است. در این حالت، پردازنده اصلی موقتاً متوقف (Stall) می‌شه تا تداخلی در داده‌ها پیش نیاد و اطلاعات خراب نشن.

```
wire CPU_MemReq_signal = (M_MemRead | M_MemWrite) & M_Valid;
wire stall_M = CPU_MemReq_signal & VPU_MemReq_signal;
wire stall_E = stall_M;
```

3.3) رفع وابستگی داده درون واحد برداری (Data Hazards)

یکی از چالش‌های مهم در پردازش موازی، وابستگی داده‌هاست. فرض کنید VPU در حال خواندن یک آرایه از حافظه است (دستور vload) و بلافاصله دستور بعدی می‌خواهد روی همان آرایه عملیات جمع انجام دهد.

برای اینکه سرعت سیستم بالا برود، ما به گونه‌ای طراحی کردیم که واحدهای حافظه و ALU در داخل VPU بتوانند همزمان کار کنند. اما اگر ALU بخواد روی عنصری کار کند که واحد حافظه هنوز آن را از رم نخوانده است، اطلاعات خراب می‌شود. برای حل این مشکل، یک مدار تشخیص وابستگی داخل VPU تعبیه کردیم. این مدار بررسی می‌کند که اگر ثبات‌های مبدا یا مقصد با هم تداخل دارند و واحد محاسبه (ALU) از واحد بارگذاری حافظه جلو افتاده است، ALU موقتاً متوقف (alu_must_stall) شود تا داده‌ی صحیح از حافظه برسد و سپس پردازش ادامه پیدا کند. این طراحی باعث می‌شود بدون ایجاد تاخیر اضافی، ایمنی داده‌ها کاملاً تضمین شود.

```

wire mem_is_vload = (mem_opcode == 6'b000010);
wire dependency_vs1 = mem_busy && mem_is_vload && (mem_vd == alu_vs1);
wire dependency_vs2 = mem_busy && mem_is_vload && (mem_vd == alu_vs2);
wire dependency_vd = mem_busy && mem_is_vload && (mem_vd == alu_vd);

```

- پیاده‌سازی مجموعه دستورات و اجرای گام‌به‌گام

4.1) مجموعه دستورات پشتیبانی شده

توی این پروژه، ما به مجموعه دستور کاربردی و استاندارد از RISC-V رو برای واحد برداری پیاده‌سازی کردیم. دستوراتی که VPU ما می‌تونه اجرا کنه اینا هستن:

- دستورات حافظه: برای خوندن متوالی یک بلوک از داده‌ها از حافظه (vload با آپکد 000010) و نوشتن گروهی عناصر توی حافظه (vstore با آپکد 000011).
- دستورات محاسباتی اعداد صحیح: شامل جمع برداری (vadd.vv با آپکد 000000) و ضرب برداری (vmul.vv با آپکد 000001) که به‌صورت موازی روی تکتک عناصر دو آرایه انجام می‌شن.
- محاسبات اعشاری: دستور جمع ممیز شناور (vfadd.vv با آپکد 000101) که برای پردازش داده‌های علمی و اعداد اعشاری کاربرد داره.

```

if (alu_opcode == 6'b000000) begin
    vector_registers[{alu_vd, alu_counter[7:0]}] <= fpu_in_a + fpu_in_b;
end else if (alu_opcode == 6'b000001) begin
    vector_registers[{alu_vd, alu_counter[7:0]}] <= fpu_in_a * fpu_in_b;
end else if (alu_opcode == 6'b000101) begin
    vector_registers[{alu_vd, alu_counter[7:0]}] <= fpu_out;
end

```

4.2) واحد ممیز شناور یا FPU

برای اینکه بتونیم اعداد اعشاری رو با دقت بالا جمع کنیم، به ماژول کاملاً مستقل به اسم FP_Adder طراحی کردیم. این ماژول دو تا عدد ۳۲ بیتی استاندارد رو به عنوان ورودی می‌گیره. روند کارش به این شکله که اول توان‌ها (Exponent) رو با هم هم‌تراز می‌کنه، بعد بخش‌های اعشاری (Fraction) رو با هم جمع یا تفریق می‌کنه، و در نهایت نتیجه رو شیفت می‌ده تا نرمال‌سازی بشه (یعنی عدد به فرم استاندارد دربیاد). خروجی این ماژول هم مستقیماً به واحد برداری وصل شده تا جواب نهایی توی همون ثبات‌های برداری که قبلاً گفتیم ذخیره بشه.

```

FP_Adder fpu_unit(
    .a(fpu_in_a),
    .b(fpu_in_b),
    .out(fpu_out)
);

```

4.3) اجرای گام‌به‌گام دستورات

اگه بخوایم مسیر اجرای یک دستور برداری (مثلاً جمع دو تا آرایه) رو خیلی ساده و قدم‌به‌قدم بررسی کنیم، داستان از این قرار می‌شه:

1. شروع عملیات: اول از همه پردازنده اصلی آدرس‌ها، طول بردار و نوع عملیات رو آماده می‌کنه. بعد سیگنال Start رو یک می‌کنه. نکته مهم اینه که به محض اینکه Start=1 می‌شه و لبه‌ی کلاک می‌خوره، مقادیر ورودی با موفقیت داخل ثبات‌ها بارگذاری می‌شن و این روند کاملاً مستقل از سیگنال ریست (Rst) کار می‌کنه.
2. درگیر شدن واحد: VPU به محض دریافت دستور، سیگنال Busy (مشغول بودن) رو روشن می‌کنه. با این کار پردازنده می‌فهمه که فعلاً نباید دستور برداری دیگه‌ای بفرسته.
3. اجرای موازی: حالا شمارنده‌های داخلی VPU (مثل alu_counter یا mem_counter) شروع به کار می‌کنن. توی هر چرخه، عناصر از ثبات‌های مبدا خونده می‌شن، عملیات روشن انجام می‌شه و نتیجه بلافاصله توی ثبات مقصد نوشته می‌شه.
4. پایان کار: وقتی شمارنده دقیقاً به اندازه طول بردار رسید (VectorLength - 1)، کار تموم می‌شه. اینجا وضعیت مشغول (Busy) خاموش می‌شه و سیگنال Done روشن می‌شه تا پردازنده اصلی بفهمه عملیات با موفقیت به پایان رسیده.

```
if (alu_counter == alu_vl - 1) begin
    alu_busy <= 0;
    VPU_Done <= 1;
end else begin
    alu_counter <= alu_counter + 1;
end
```

• ارزیابی عملکرد و تحلیل نتایج

5.1 سناریوی آزمون و تست‌بنچ

یه فایل تست‌بنچ جامع (tb.v) طراحی کردیم. روند کار توی این تست‌بنچ به این صورته که اول حافظه‌ی دستورات و داده‌ها مقداردهی می‌شن و ما یه آرایه بزرگ به طول ۲۵۶ عنصر رو برای پردازش آماده می‌کنیم. سناریوی تستی که در نظر گرفتیم شامل جمع دو آرایه، ضرب اون‌ها و یه عملیات جمع ممیز شناور (اعشاری) هست. برنامه ما طوری نوشته شده که اول این عملیات‌ها رو به شکل سنتی و اسکالر روی CPU اجرا می‌کنه و تعداد چرخه‌های کلاک رو توی متغیر scalar_cycles ذخیره می‌کنه. بعد همون کار رو می‌سپاره به واحد برداری و زمانش رو توی vector_cycles ثبت می‌کنه.

در نهایت، برای اینکه خیالمون راحت باشه که سرعت بیشتر باعث خطای محاسباتی نشده، یه بخش اعتبارسنجی (Verification) نوشتیم که مقادیر ذخیره شده تو حافظه رو چک می‌کنه تا مطمئن بشیم جواب‌های استخراج شده با مقادیر مورد انتظارمون دقیقاً یکسان هستن. تصویر بخشی از آن:

```
for(i=0; i<256; i=i+1) begin
    if(DataMem[512+i] != (i+1)*77 || DataMem[768+i] !=
        $display("ERROR at Add Index %0d: Expected %0d
end

for(i=0; i<256; i=i+1) begin
    if(DataMem[1024+i] != (i+1) * ((i+1)*10))
        $display("ERROR at Mul Index %0d: Expected %0d
end
```

5.2 استخراج نتایج زمانی

برای مقایسه عملکرد، ما باید دقیقاً می‌فهمیدیم هر روش چقدر زمان (بر حسب چرخه کلاک) نیاز داره. به همین خاطر، توی تست‌بنچ یه متغیر به اسم total_cycles تعریف کردیم که با هر کلاک، یکی بهش اضافه می‌شه.

روند ثابت زمان به این شکله که وقتی اجرای کدهای اسکالر (روی پردازنده اصلی) تموم می‌شه، مقدار این شمارنده رو توی `scalar_cycles` ذخیره می‌کنیم. بعد از اون، برنامه وارد فاز پردازش برداری می‌شه و وقتی اونم تموم شد، زمان اجرای برداری رو محاسبه کرده و توی `vector_cycles` می‌ریزیم. این روش به ما یه عدد دقیق و واقعی از زمان اجرای هر دو حالت می‌ده.

5.3 محاسبه و تحلیل Speedup

حالا که زمان اجرای هر دو حالت رو داریم، باید ببینیم این VPU چقدر سیستم رو سریع‌تر کرده. برای این کار، همون‌طور که توی صورت پروژه خواسته شده بود، از رابطه ریاضی زیر استفاده کردیم:

$$Speedup = \frac{ExecutionCycles (scalar)}{ExecutionCycles (vector)}$$

در کد تست‌بنچ دقیقاً همین فرمول رو پیاده‌سازی کردیم تا بعد از پایان شبیه‌سازی، عدد نهایی افزایش سرعت (Speedup) رو به صورت خودکار روی صفحه نمایش (کنسول) چاپ کنه.

خروجی ترمینال:

```
> vvp a.out

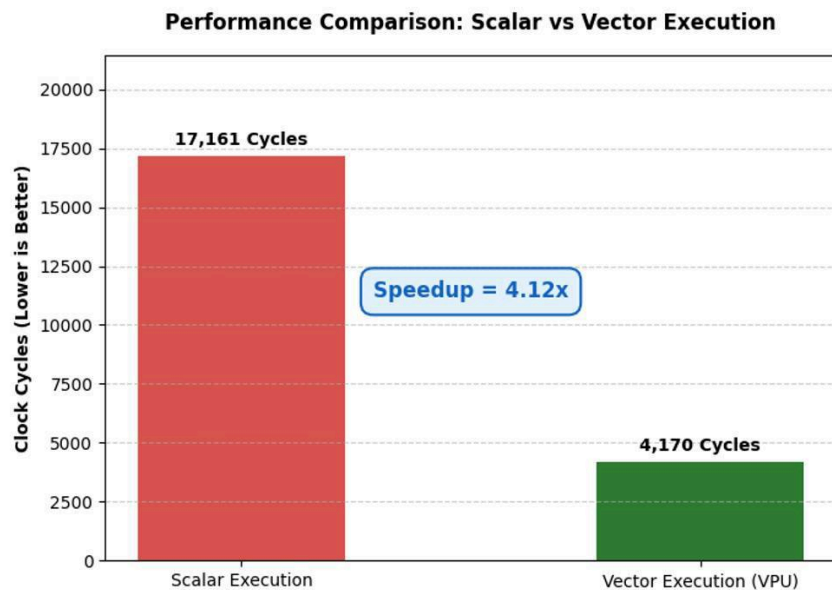
=====
#          BENCHMARK & VERIFICATION RESULTS
=====
-----
#  Scalar Execution Cycles : 17161
#  Vector Execution Cycles : 4170
#  SPEEDUP (Scalar / Vector) : 4.115348
=====

tb.v:177: $finish called at 213315 (1s)
```

(پ.ن: شکل موج در انتهای گزارش قرار گرفته است.)

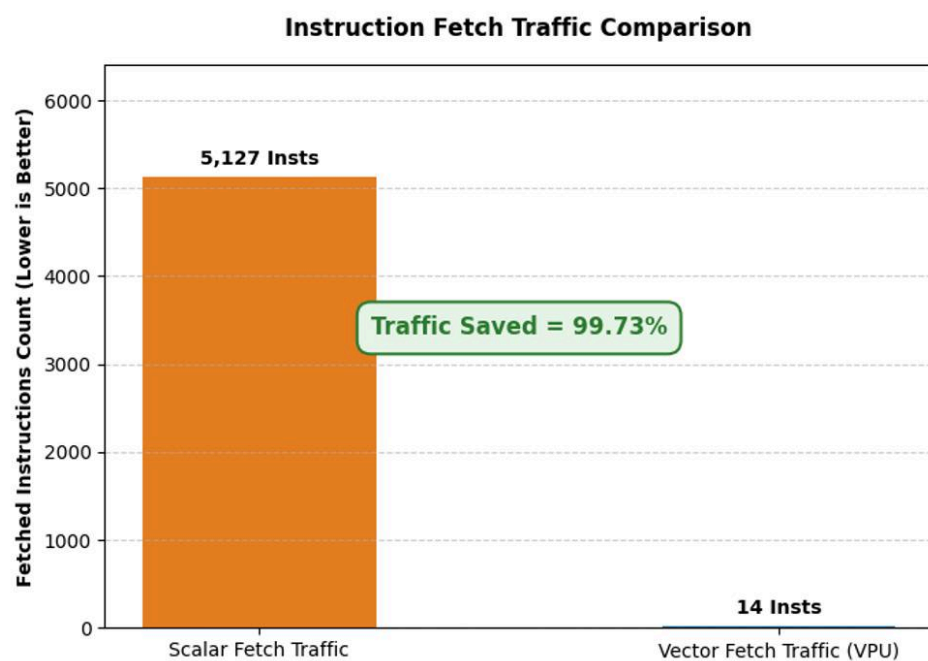
● مقایسه عملکردی

6.1 مقایسه تعداد کلاک سایلک ها:



همانطور که در نمودار مشخص است، تعداد سایکل ها تقریباً تقریباً 0.25 برابر شده که این یعنی ما افزایش سرعت 4 برابری داریم.

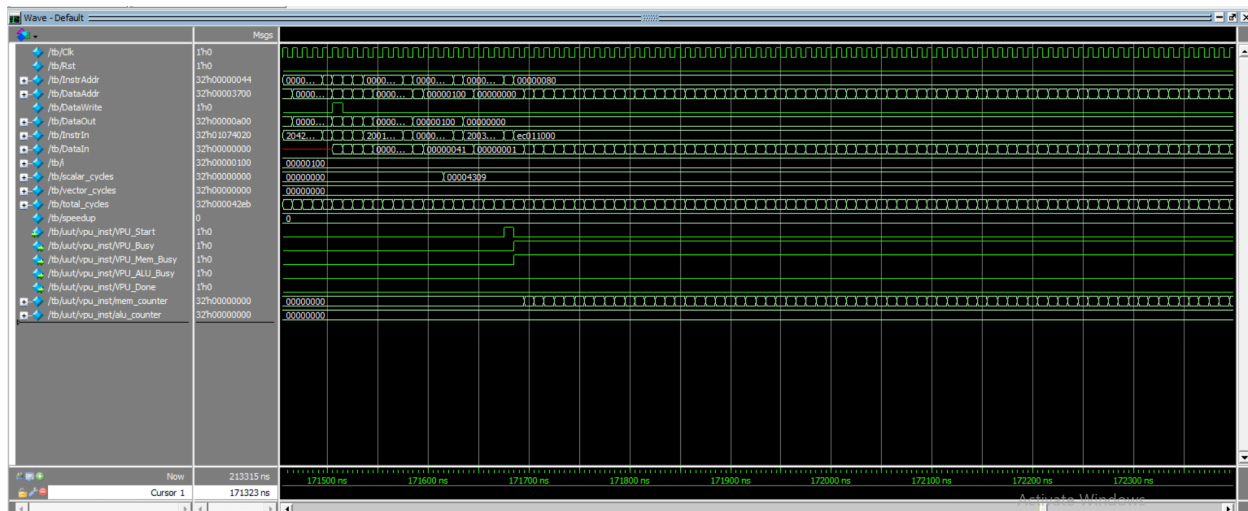
مقایسه ی تعداد دستورات دریافتی:



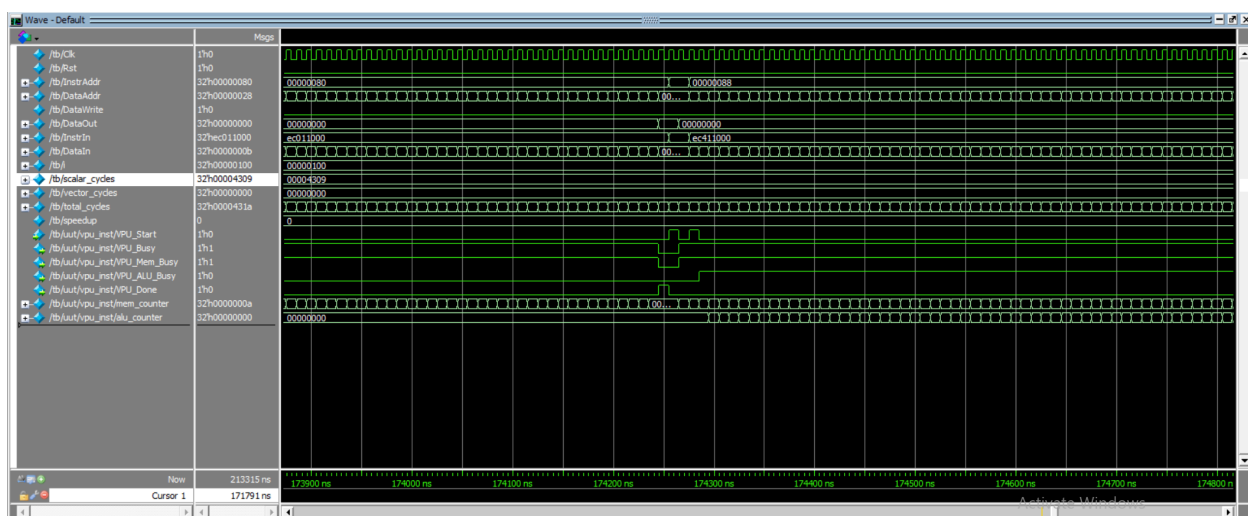
تعداد دستورات دریافتی در VPU فقط 14 تاست، ولی قبل از استفاده از VPU تعداد دستورات حدود 5000 تا بود!!

● شکل موج

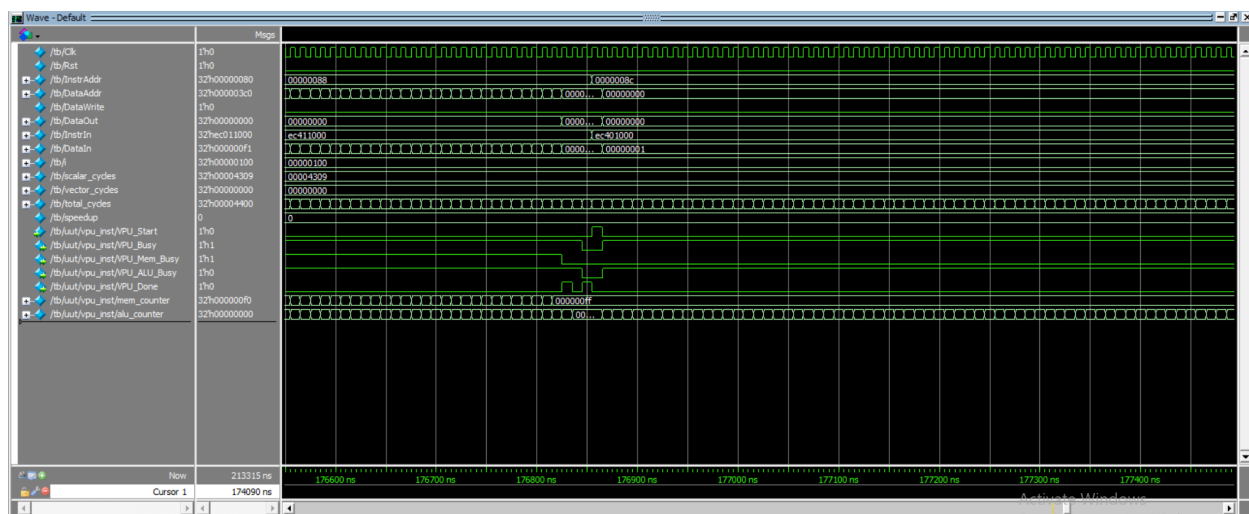
7.1 لحظه صدور اولین دستور برداری توسط CPU و فعال سازی واحد VPU



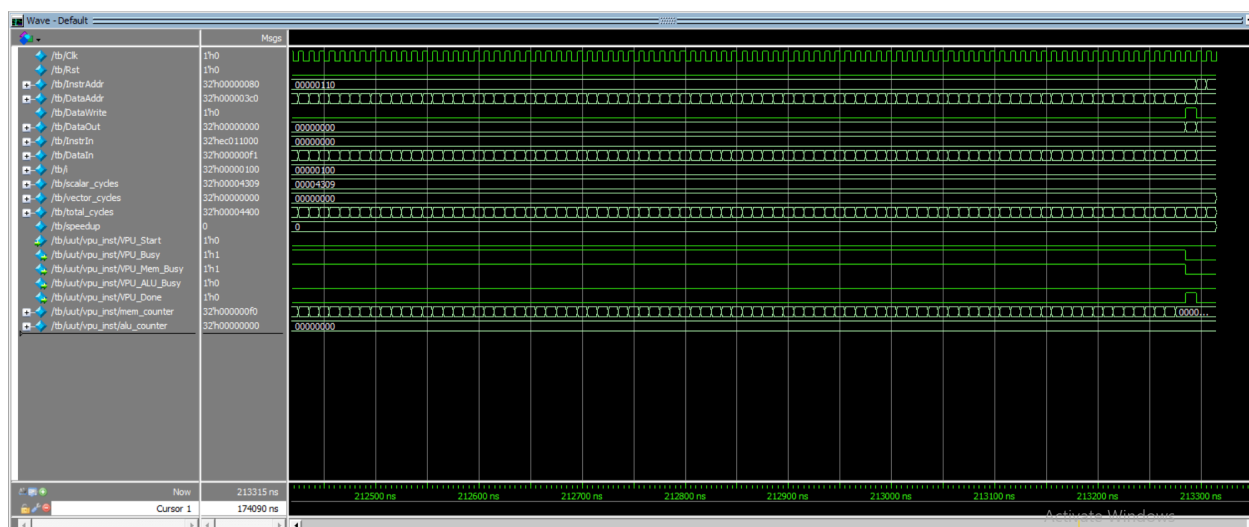
7.2) روند واکنشی خودکار داده‌ها از حافظه توسط کنترل‌کننده مستقل VPU



7.3) روند اجرای محاسبات برداری روی عناصر آرایه



7.4) صدور سیگنال اتمام کار (VPU_Done) و آزدسازی پردازنده اصلی



توضیح درباره قسمت آخر تصویر:

متغیرهای محاسباتی بنچمارک، (vector_cycle، scalar_cycle و speedup) متغیرهای نرم‌افزاری/تست‌بنچ از نوع Real هستند و ثبات سخت‌افزاری محسوب نمی‌شوند. محاسبات مقدار Speedup مستقیماً در انتهای زمان شبیه‌سازی و هم‌زمان با دستور finish\$ انجام می‌پذیرد. از آنجا که با اجرای دستور finish\$ در زمان ، موتور شبیه‌ساز زمان‌بندی را متوقف می‌سازد، فرصت بازنویسی (Refresh) خط زمان‌بندی این متغیر روی تصویر موج فراهم نمی‌شود و مقدار آن در گام زمانی آخر به صورت پیش‌فرض باقی می‌ماند. با این حال مقدار دقیق آن در نتیجه نهایی مشخص است.

